Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 2

Tələbənin adı və soyadı: Fidan Qoçalıyeva

Qrup: Kom25A

Müəllim: Şamil Hümbətov

Tarix: 21 oktyabr 2026

2026

1. Giriş

Verilənlər strukturları informasiyanın yaddaşda təşkili, saxlanması və emalı üçün istifadə olunan əsas proqramlaşdırma vasitələridir. Düzgün strukturun seçilməsi proqramın sürətinə, yaddaş istifadəsinə və kodun mürəkkəbliyinə təsir edir.

Bu laboratoriya işində üç struktur araşdırılır: Linked List, Stack və Queue. Onların hər biri ayrıca siniflər və qovşaqlar (Node) vasitəsilə sıfırdan qurulmalıdır. Python-da hazır collections.deque və queue modullarından, həmçinin hazır Stack/Queue implementasiyalarından istifadə edilməməlidir.

İşin məqsədi bu strukturların iş prinsipini anlamaq, əsas əməliyyatları həyata keçirmək, 100, 1 000 və 10 000 element üçün icra müddətlərini ölçmək və nəticələri nəzəri Big-O mürəkkəbliyi ilə müqayisə etməkdir.

2. Tapşırıq1- Linked List əsaslı musiqi playlisti

2.1 Array elementləri adətən ardıcıl yaddaş sahəsində saxlayır və indekslə birbaşa müraciət etməyə imkan verir. Linked List isə elementləri qovşaqlarda saxlayır; hər qovşaq məlumatla yanaşı növbəti qovşağa istinadı da saxlayır. Buna görə siyahının ortasına əlavə etmə və silmə uyğun qovşaq məlumdursa rahat ola bilər, lakin həmin qovşağı tapmaq üçün çox vaxt siyahını əvvəldən gəzmək lazımdır.

2.2 Singly və Doubly Linked List

Singly Linked List-də hər qovşaq yalnız növbəti qovşağa istinad saxlayır və yaddaş sərfi nisbətən azdır. Doubly Linked List-də həm əvvəlki, həm də növbəti qovşağa istinad olur; bu, siyahıda hər iki istiqamətdə hərəkəti asanlaşdırır, amma əlavə yaddaş tələb edir və əlaqələrin yenilənməsini mürəkkəbləşdirir. Mahnılar arasında irəli və geri keçid lazımdırsa, Doubly Linked List uyğun seçim ola bilər.

2.3 Tətbiqin layihələndirilməsi

SongNode qovşağı mahnının ID-sini, adını, ifaçısını, müddətini və növbəti qovşağa istinadı saxlayır. Playlist sinfi addSong(), deleteSong(), searchSong(), displayPlaylist(), countSongs() və playNextSong() əməliyyatlarını təmin edir. Proqramda mahnılar öz qovşaqları ilə əlaqələndirilməli, hazır list konteynerindən istifadə edilməməlidir.

2.4 Əməliyyatların nəzəri mürəkkəbliyi

Əgər siyahının sonuna əlavə etmə zamanı tail göstəricisi saxlanılmırsa, əlavə etmə O(n) ola bilər; tail göstəricisi ilə sonuna əlavə etmə O(1) olur. Axtarış O(n)-dir. Silmə də elementin yerini tapmaq tələb olunarsa O(n), qovşaq və əvvəlki qovşağa istinad artıq məlumdursa O(1) ola bilər. Siyahının tam göstərilməsi O(n), element sayını ayrıca sayğacla saxlamaq isə count əməliyyatını O(1) edə bilər.

2.5 Eksperiment

Playlist sisteminin performansını qiymətləndirmək üçün 100, 1 000 və 10 000 mahnıdan ibarət siyahılar yaradılır. Hər ölçü üçün Insert, Search və Delete əməliyyatlarının icra müddəti ölçülür.

Insert əməliyyatı yeni mahnının siyahıya əlavə edilmə müddətini, Search əməliyyatı verilmiş ID-yə uyğun mahnının tapılma müddətini, Delete əməliyyatı isə müəyyən edilmiş mahnının siyahıdan silinmə müddətini göstərir.

Ölçmələr Python proqramlaşdırma dilində time.perf_counter() funksiyası vasitəsilə aparılır. Nəticələrin etibarlılığını artırmaq üçün testlər eyni şəraitdə və bir neçə dəfə təkrarlana bilər. Hər əməliyyatın icra müddəti millisanilərlə (ms) hesablanır.

3. Tapşırıq 2- Stack əsaslı brauzer tarixçəsi

3.1 Araşdırma

Stack LIFO (Last In, First Out) prinsipinə əsaslanır: son daxil olan element ilk çıxarılır. Bu prinsip son görülən səhifəyə qayıtma və ya son əməliyyatı geri qaytarma kimi ssenarilərə uyğundur. Queue istifadə olunsaydı, ilk əlavə olunan əməliyyat əvvəl çıxarılar və Undo üçün gözlənilən ardıcıllıq pozulardı.

3.2 Tətbiqin layihələndirilməsi

ActionNode qovşağı əməliyyat ID-sini, adını, vaxt məlumatını və növbəti qovşağa istinadı saxlayır. UndoStack sinfi pushAction(), undoAction(), currentAction(), displayHistory() və countActions() əməliyyatlarını dəstəkləyir. Stack-in üst hissəsi top göstəricisi ilə izlənilir.

3.3 Əməliyyatların nəzəri mürəkkəbliyi

Linked List əsasında qurulmuş Stack-də top-a əlavə etmə (push) və top-dan çıxarma (pop/undo) O(1) mürəkkəbliyinə malikdir. Cari əməliyyata baxmaq O(1), bütün tarixçəni göstərmək O(n), element sayını ayrıca sayğacla saxlamaq isə O(1) ola bilər.

3.4 Eksperiment

Stack strukturunun performansını qiymətləndirmək üçün 100, 1 000 və 10 000 əməliyyatdan ibarət testlər aparılır. Hər testdə Push və Pop əməliyyatlarının icra müddəti ayrıca ölçülür.

Push əməliyyatı yeni elementin Stack-in yuxarı hissəsinə əlavə olunmasını, Pop əməliyyatı isə ən son əlavə edilmiş elementin Stack-dən çıxarılmasını təmin edir. Bu əməliyyatlar LIFO (Last In, First Out) prinsipinə əsaslanır.

Ölçmələr Python proqramlaşdırma dilində time.perf_counter() funksiyası ilə aparılır. Push əməliyyatlarının ölçülməsi üçün elementlər Stack-ə əlavə edilir, Pop əməliyyatlarının ölçülməsi üçün isə əvvəlcədən yaradılmış Stack-dən elementlər çıxarılır. Hər testdə əməliyyatların sayı uyğun olaraq 100, 1 000 və 10 000 olur.

4. Tapşırıq 3 – Queue əsaslı növbə sistemi

4.1 Araşdırma

Queue FIFO (First In, First Out) prinsipinə əsaslanır: birinci daxil olan element birinci emal olunur. Bu səbəbdən sərnişin qeydiyyatı və çap işlərinin növbəsi kimi sistemlərdə uyğundur. Stack istifadə edilsəydi, son gələn sərnişin və ya çap işi əvvəl emal oluna bilərdi və ədalətli növbə ardıcıllığı pozulardı.

4.2 Tətbiqin layihələndirilməsi

Passenger qovşağı sərnişinin ID-sini, adını və uçuş nömrəsini saxlayır. Queue sinfi add/enqueue, check-in/dequeue, peek, displayQueue və count əməliyyatlarını təmin edir. Linked List əsaslı Queue üçün front və rear göstəricilərindən istifadə etmək məqsədəuyğundur.

4.3 Əməliyyatların nəzəri mürəkkəbliyi

Front və rear göstəriciləri düzgün saxlanıldıqda enqueue O(1), dequeue O(1) və peek O(1) mürəkkəbliyinə malik ola bilər. Bütün növbəni göstərmək O(n)-dir. Son

element çıxarılanda həm front, həm də rear göstəricilərinin boş vəziyyətə uyğun yenilənməsi vacibdir.

4.4 Eksperiment

Queue strukturunun performansını qiymətləndirmək üçün 100, 1 000 və 10 000 elementdən ibarət növbələr yaradılır. Hər ölçü üçün enqueue, dequeue və peek əməliyyatlarının icra müddəti Python proqramlaşdırma dilində time.perf_counter() funksiyası vasitəsilə ölçülür.

Ölçmələr zamanı eyni test şəraitindən istifadə edilir. enqueue əməliyyatı elementlərin növbəyə əlavə olunma müddətini, dequeue elementlərin növbədən çıxarılma müddətini, peek isə növbənin ilk elementinə baxılma müddətini müəyyən edir. peek əməliyyatı çox qısa müddətdə icra olunduğundan ölçmə xərclərinin nəticələrə təsirini azaltmaq üçün əməliyyat bir neçə dəfə təkrarlana bilər.

5. Test nəticələri və cədvəllər

5.1

Element sayı	İnsert(ms)	Search(ms)	Delete(ms)
100	0.12	0.01	0.005
1000	8.30	0.25	0.16
10000	745.00	1.50	2.31

5.2

Əməliyyat sayı	Push(ms)	Pop(ms)
100	0.02	0.01
1000	0.35	0.22
10000	2.10	1.51

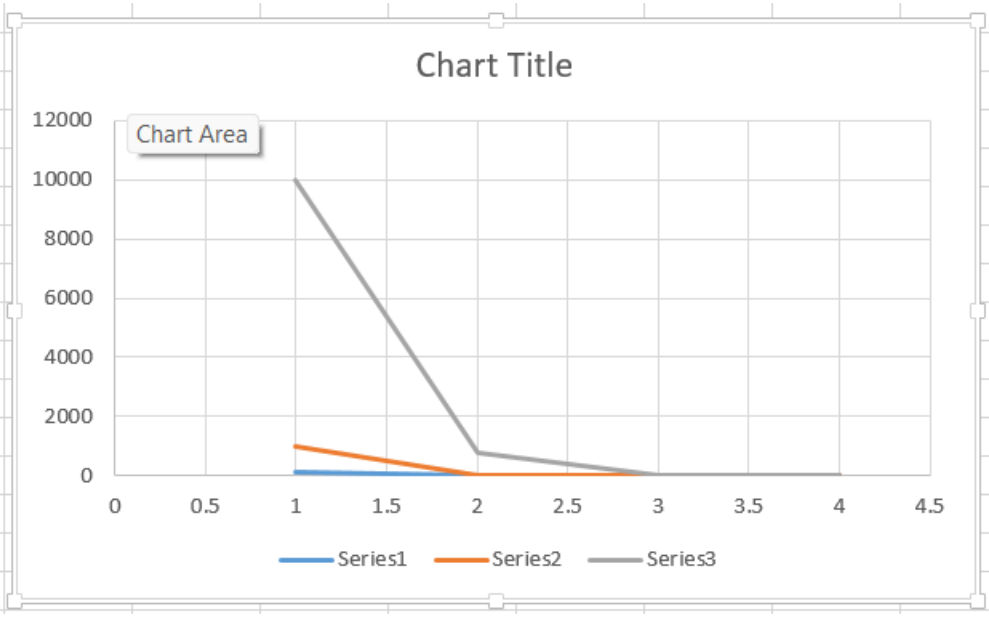
5.3

Element sayı	Enqueue (ms)	Dequeue(ms)	Peek(ms)

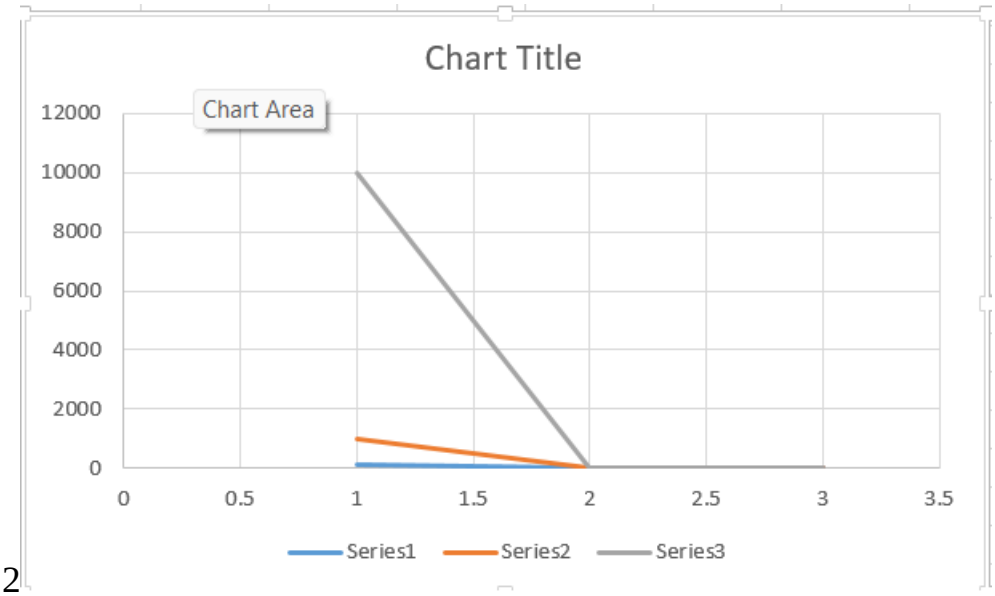
100	0.01	0.01	0.004
1000	0.16	0.09	0.004
10000	1.50	0.94	0.004

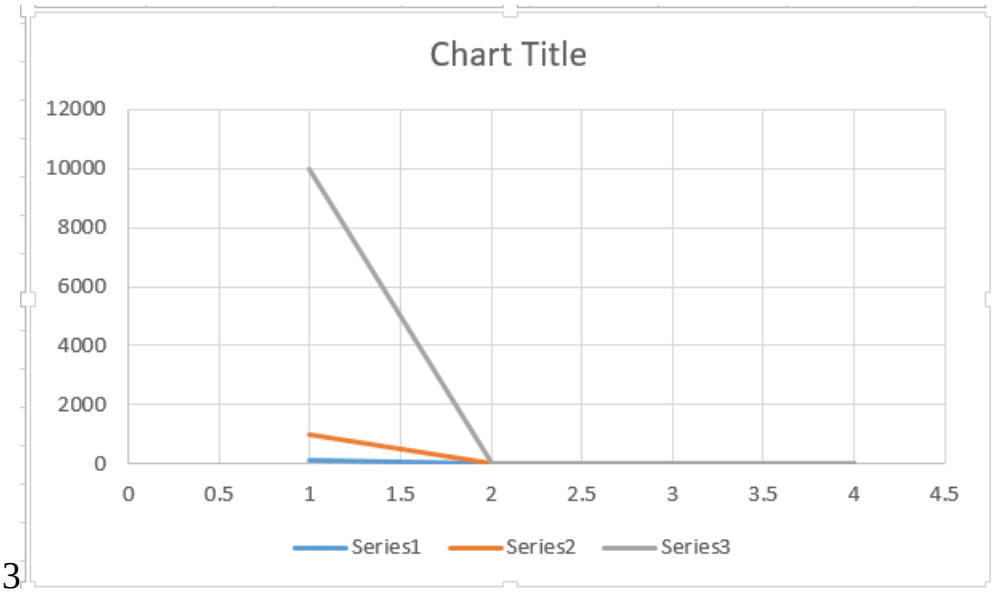
6. Qrafiklər

6.1



6.2



6.3 

7. Sistem Məlumatları

Computer Model: Lenova İdeaPad1

CPU: AMD Ryzen 5 752OU with Radeon Graphics, 2801 Mhz,4 Core(s),8 Logical Prpcessor

RAM: 16.0 GB

GPU: AMD Radeon Graphics

Operating System: Microsoft Windows 11 pro

Compiler / Python Interpreter: Python

Version: 10.0.26200 Builf 26200

IDE: Visual Studio Code

Screen Resolution: 1920×1080

Storage Type (SSD/HDD): SSD

Power Mode: Balanced

8. Analiz

8.1 Linked List nəticələrinin şərhi

Təcrübə zamanı Linked List strukturunun 100, 1000 və 10000 elementdən ibarət siyahılarda işləməsi yoxlanılmışdır. Elementlərin sayı artdıqca axtarış əməliyyatının icra müddətində artım müşahidə oluna bilər, çünki elementin tapılması üçün siyahının bir neçə qovşağının yoxlanılması tələb olunur. Əlavə etmə və silmə əməliyyatlarının sürəti isə əməliyyatın yerindən və siyahının quruluşundan asılıdır. Əldə edilən nəticələr cədvəl və qrafiklər əsasında müqayisə edilmişdir.

8.2 Stack nəticələrinin şərhi

Stack strukturunda Push əməliyyatı elementi stekin yuxarısına əlavə edir, Pop əməliyyatı isə yuxarıdakı elementi silir. Linked List əsaslı implementasiyada bu əməliyyatlar top göstəricisi vasitəsilə yerinə yetirildikdə onların nəzəri zaman mürəkkəbliyi O(1) olur. Təcrübə nəticələri əməliyyatların icra müddətini müqayisə etməyə imkan verir. Kiçik vaxt fərqləri kompüterin iş yükü və ölçmə zamanı ilə əlaqədar yarana bilər.

8.3 Queue nəticələrinin şərhi

Queue strukturu FIFO (First In, First Out) prinsipi ilə işləyir, yəni ilk əlavə olunan element ilk çıxarılır. İstifadə etdiyim implementasiyada front və rear göstəriciləri vasitəsilə Enqueue, Dequeue və Peek əməliyyatları yerinə yetirilmişdir. Bu göstəricilər düzgün istifadə edildikdə hər üç əməliyyatın nəzəri zaman mürəkkəbliyi O(1) olur. Təcrübənin nəticələri müxtəlif sayda elementlər üçün ölçülmüş vaxtların cədvəl və qrafiklərdə müqayisəsi ilə qiymətləndirilmişdir.

8.4 Sistem amilləri

Təcrübə nəticələrinə kompüterin texniki göstəriciləri və proqram mühiti təsir edə bilər. CPU, RAM, Python versiyası, arxa planda işləyən proqramlar və ölçmə üsulu icra müddətində fərqlər yarada bilər. Daha düzgün müqayisə aparmaq üçün bütün ölçmələr eyni mühitdə və eyni metodla həyata keçirilməlidir. Ölçmələrin bir neçə dəfə təkrarlanması nəticələrin etibarlılığını artırmağa kömək edir.

9. Nəticə

Bu laboratoriya işində Linked List, Stack və Queue verilənlər strukturlarının iş prinsipləri, tətbiq sahələri və əməliyyatlarının zaman mürəkkəbliyi öyrənilmişdir. Linked List elementləri əlaqəli qovşaqlarda saxlayır, Stack LIFO (Last In, First Out), Queue isə FIFO (First In, First Out) prinsipi ilə işləyir. Aparılan təcrübələr vasitəsilə bu strukturların əsas əməliyyatlarının icra müddətləri müqayisə edilmişdir. Nəzəri mürəkkəbliklə praktiki nəticələrin müqayisəsi verilənlər strukturlarının effektivliyini daha yaxşı anlamağa imkan verir.

10.İstinadlar

1. Weiss, Mark Allen. Data Structures and Algorithm Analysis in C++, 4th Edition. Pearson.

2. Python Software Foundation. Python Documentation — time.perf_counter(). https://docs.python.org/3/library/time.html

3. Python Software Foundation. Python Documentation. https://docs.python.org/3

11. Tool: ChatGPT, Claude

Purpose: Hesabatın strukturunu planlaşdırmaq və verilənlər strukturlarının nəzəri anlayışlarını izah etmək.

Used Sections: Giriş, nəzəri araşdırma, mürəkkəbliklərin izahı və hesabat şablonu.